DREW

Lecture 12: The von Neumann Model

CSCI 260 Computer Systems & Architecture
Fall 2024

Andy Siyuan Feng

09/25/2024

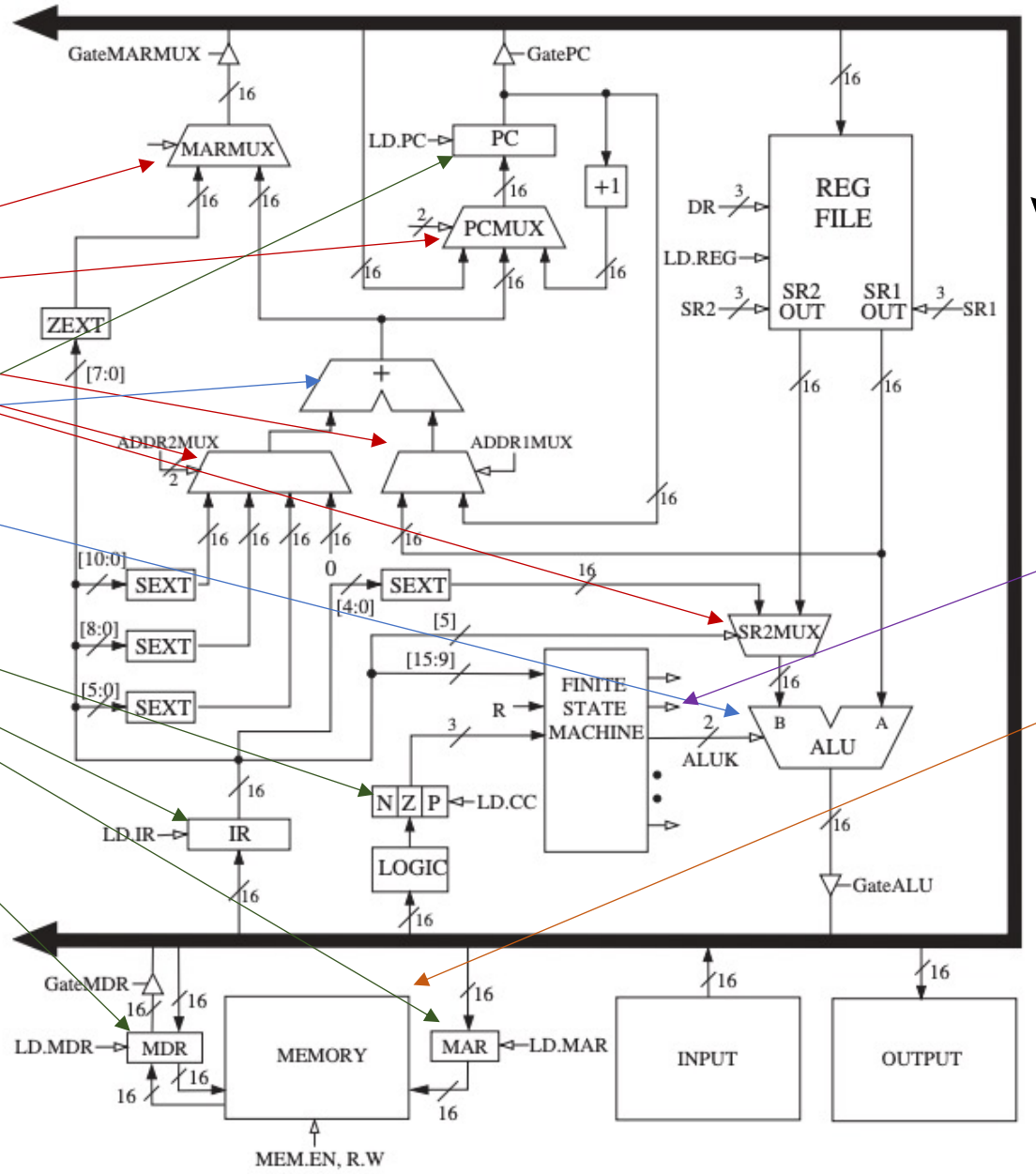
§3.7 Data Path of the LC-3: Where are we now?

- Little-Computer 3

MUX

Adder/ALU

Register



Bus

FSM

Memory

DREW

Thus far,

John von Neumann, public domain



- Talked about different components in a modern computer:
  - Combinational Logic: Decision making units
  - Storage units
  - Sequential Logic: both decision and storage

- Now, put everything together. How?

- The *von Neumann model* [1946] provided the fundamental structure of a computer for processing programs.
  - **Recall**: a general-purpose computer can take not only the arguments of a program as input, but also programs as input.

§4.1 The von Neumann Model

The model is consisted of 5 components:

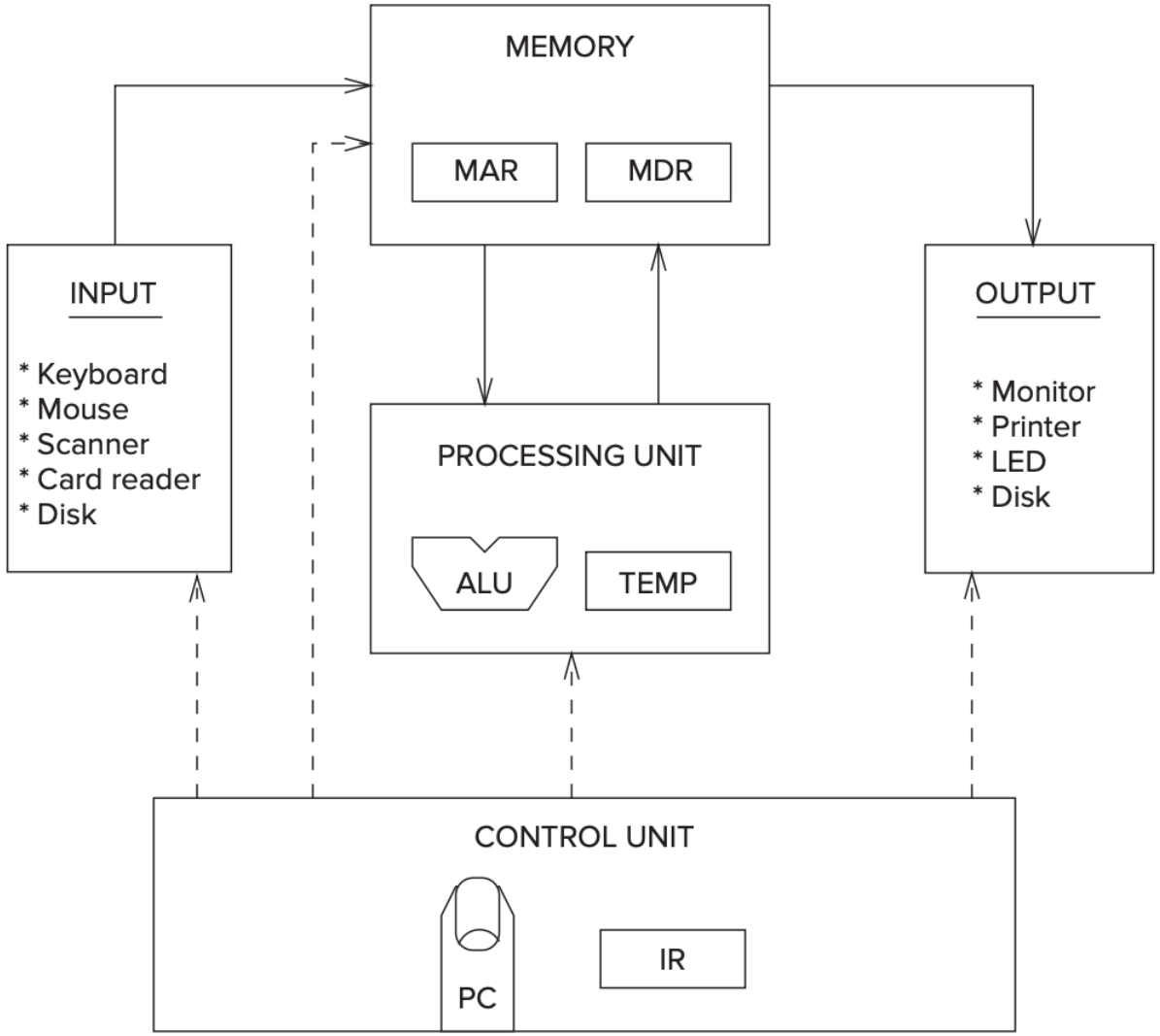
1. *Memory*: contains data, as well as programs in the form of *instructions*.
2. a *Processing Unit*: the actual part performing operations; e.g., the *ALU*.
3. Input
4. Output
5. a *Control Unit*: for interpreting and keeping track of instructions.

- Nowadays, have more specialized units of various kind, but at its core, still the von Neumann model.

The von Neumann Model

Consisted of 5 components:

1. *Memory*
2. Processing Unit
3. Input
4. Output
5. Control Unit



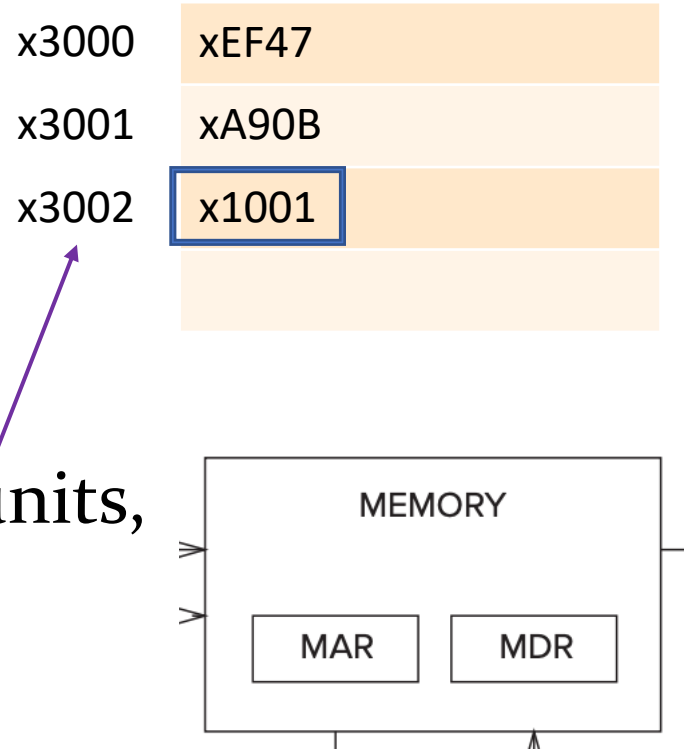
DREW

Memory

- **Recall**: memory is a $2^k \times m$ array, where there are 2^k address locations with m-bit addressability.

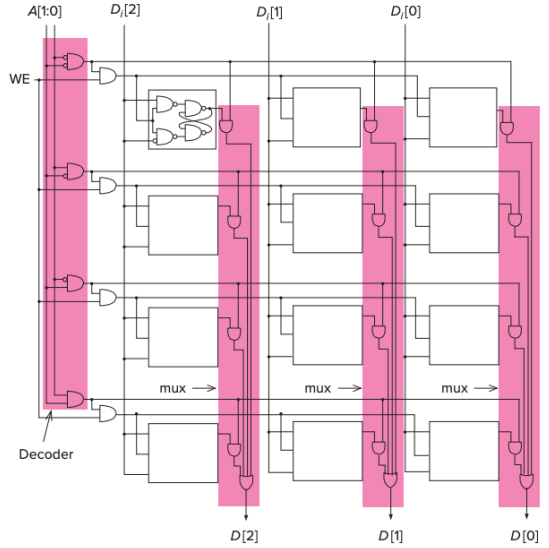
- 2 operations for memory
  - *LOAD* from memory: read value from a location.
  - *STORE* to memory: write value to a location.

- To perform these operations and interact with other units, employ 2 registers as the interface for memory.
  - *MAR* (Memory Address Register): holds a location
  - *MDR* (Memory Data Register): holds the data read from/write to the location



MAR & MDR

Note: MDR is the one who helps set up the decoder.



- To perform a *LOAD*

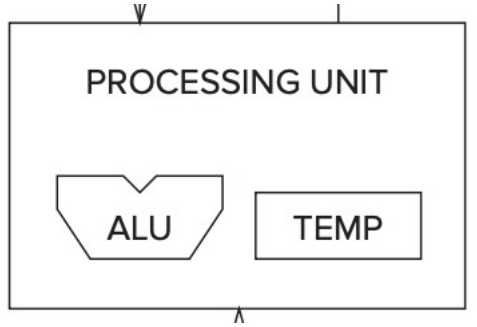
1. Write the address to MAR.
2. By *decoding*, memory goes to the address, read from it, and write its data to MDR.
3. Other units read from MDR.

- To perform a *STORE*

- Write the address to MAR and data to MDR.
- By *decoding* with a "write" signal (WE set to 1), memory goes to the address and write what's in MDR.

Processing Unit

- *ALU* (Arithmetic Logic Unit): the actual unit performing operations; e.g., ADD, AND, NOT, etc.
  - Nowadays, can have dedicated unit for MULT, SQRT, etc.
  - *Word Size*: width of the ALU, number of bits in one instruction.
    - This is 16-bit in LC-3

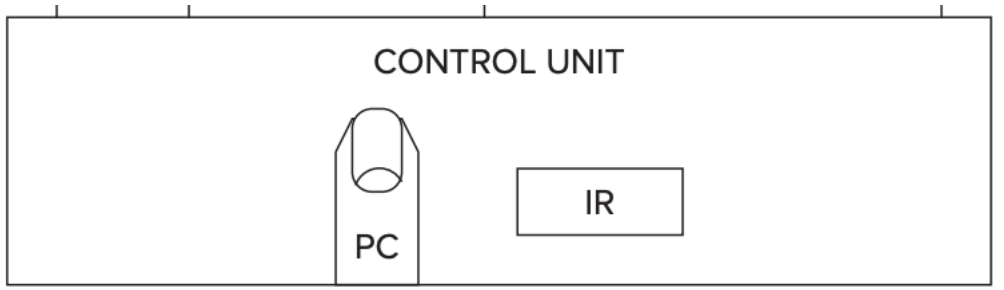


- (TEMP) *registers* for holding values temporarily.
  - Have the same width as the ALU.
  - Hence, small as storage units, but extremely fast (the closest to ALU).
    - R0~R7 in LC-3
- **Ex.**: To compute $(x + y) \cdot z$, we don't want to first STORE the sum to memory and then immediately LOAD it back, considering how slow this could be.

Input/Output

- Usually treated as external components of a computer whereas the others are internal.
- Every I/O device has its own interface (similar to MAR and MDR for memory).
- Will talk about I/O and Interrupts later.

Control Unit



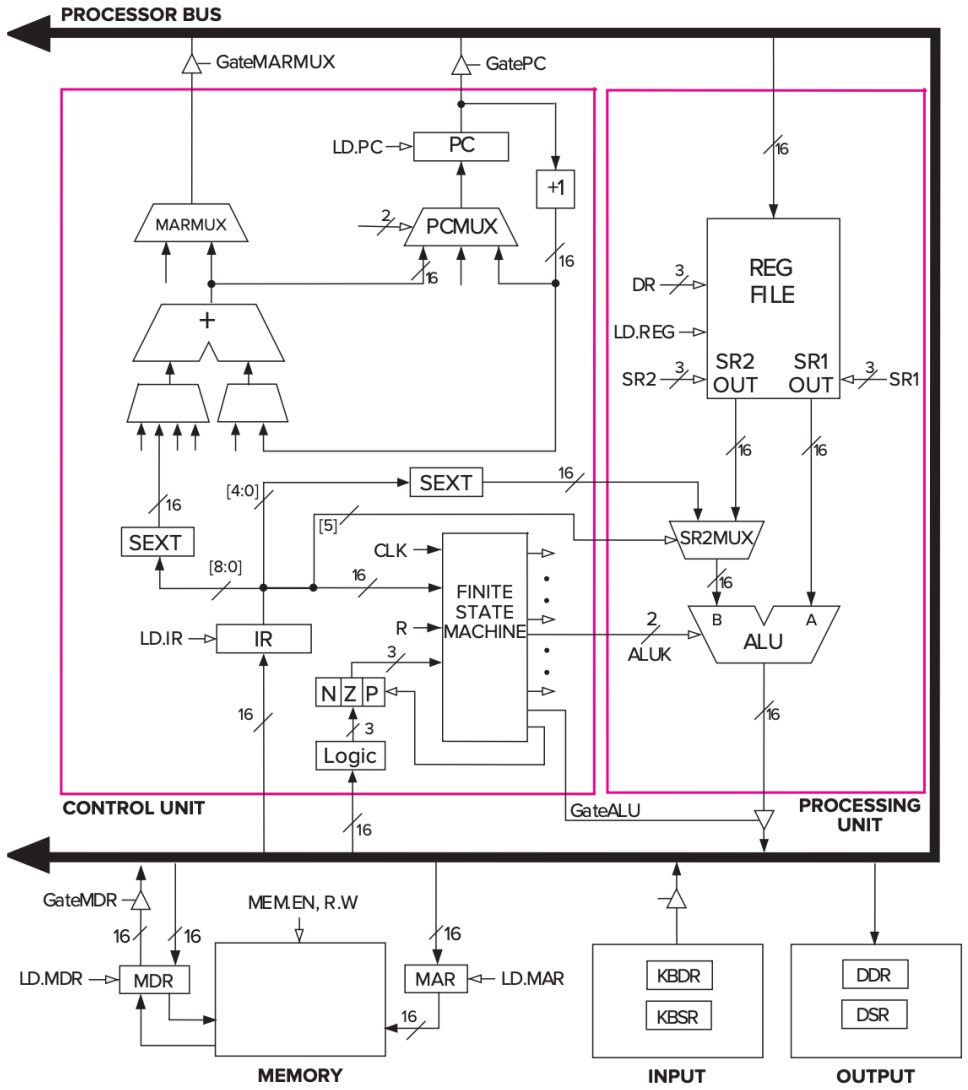
- Manages the execution of the program:
  - What's the current instruction?
  - Where is the next instruction?
  - Any preparations for the next instruction?
- *IR* (Instruction Register) contains the current instruction.
- *PC* (Program Counter) is pointing towards the memory location containing the next instruction.

- The Control Unit tells other units what to do when an instruction is "fetched" from memory and is "decoded".

DREW

The LC-3 Data Path

- Lc-3 is at core the von Neumann's model.
  - Memory with MAR & MDR
  - Input
  - Output
  - Processing unit where?
    Look for the ALU!
  - Control unit where?
    Look for the IR, PC!



§4.3 Instructions

- What is an instruction after all? Remember, this is within a computer.
- So, binary sequences, with special patterns.

- **Ex.**: Given pattern 0001 (*opcode*) as the leftmost 4 bits in a 16-bit string, this is an instruction for ADD. What are being added (*operands*) and how depends on the other 12 bits or so.

0001	110	010 0 00 110		or
Opcode		*Operands*		
ADD	R6	R2	R6	i.e., R6=R2+R6

Instructions

0001	110 010 0 00 110	or
Opcode	*Operands*	
ADD	R6 R2 R6	i.e., R6=R2+R6

- Instructions are the most basic/low-level form of program.
- Recall, all programs can ultimately compute the same thing.

- The full collection of instruction is the *ISA* (Instruction Set Architecture).
  E.g., Intel x86 (386, 485, 586 or Pentium), ARM (Apple M1/M2), PowerPC, MIPS, etc.

The Instruction Cycle

- Then, for each instruction, how is it processed within the machine?
- The 6-phase sequential *Instruction Cycle* that the control unit help orchestrates.
  1. **Fetch**: Load next instruction form the memory to the IR.
  2. **Decode**: Determine the action of the instruction.
  3. **Evaluate address**: Compute the address(es) of operands (if any).
  4. **Fetch Operands**: Load operand(s) from their address(es)/registers.
  5. **Execute**: Carry out the computation.
  6. **Store result**: Write result of instruction to destination.

Back to Phase 1 for the next instruction.
What is this? A Finite State Machine!

FETCH

The *Instruction Cycle*:

1. **Fetch**: Load next instruction form the memory to the IR.

   - Remember, that PC holds the address of the next instruction; so, load MAR with what's in the PC.
   - Then go to that location; and put the content in that location, the instruction, to MDR. [**Note**: a memory location can hold either data, instruction, or address.]
   - Load the IR with what's in MRD.

MAR ← PC; PC ← PC+1;
MRD ← Mem[MAR];
IR ← MDR;

FETCH

One important notion, how fast is our computer?

- Essentially, depends on how many instructions can execute: *Clock Cycle*.
  - 3.11 GHz CPU has 3.11 billion clock cycles (pulses) per second.

- However, also depends on how many clock cycles each instruction will take.

Step 1. MAR ← PC; PC ← PC+1; 1 clock cycle

Step 2. MRD ← Mem[MAR]; May take more than 1 clock cycle, based on R(andom)AM access time.

Step 3. IR ← MDR; 1 clock cycle

Decode

0001	110	010 0 00 110		or
Opcode		*Operands*		
ADD	R6	R2	R6	i.e., R6=R2+R6

2. **Decode**: Determine the action of the instruction. [1 clock cycle]

- *By Opcodes*: 0001 for ADD 0101 for AND 1001 for NOT
- How? It's in the name. A 4-to-16 decoder
- Once a pattern (opcode) is found, exactly one output port gives 1.
- Then, according to which output line is lighted up/opcode is identified, look for operands in the remaining bits.
- For a particular ISA, all the instructions should have the same opcode width; in LC-3, that width is 4 bits.

Evaluate Address

3. **Evaluate Address**: Compute the address(es) of operands (if any).
   - Different types of operands: an operand can be [1 clock cycle]

   a) an *immediate* number: ADD R1, R1, x5;
   b) a *register*: AND R2, R3, R4;

   [For the above two cases, we don't need to compute address, as there's no memory access at all.]

   c) or an address (also different ways the address is represented.):
   LD R2, x2A;

   [Now, we will need to compute the *Effective Address* (EA) that's being loaded from; in this case PC+x2A.]

Fetch Operands and Execute

4. **Fetch Operands**: Load operand(s) from their address(es)/registers.
   - If an *EA* was computed in Phase 3, then fetch from the EA as

     MAR ← EA;

     MDR ← Mem[MAR]; [Can be done with **Execute**.]

5. **Execute**: Carry out the computation. [1 clock cycle]

   a) For ADD, NOT, AND, send operands to ALU with the corresponding operation signaling.

   b) Modifying PC if having a control instruction; e.g., *JMP*, *BR*, etc.

   c) Nothing is done, if we are moving data; e.g., LD, ST, etc.

Store Result and repeat

6. **Store Result**: Write result of instruction to destination, memory or register.

- Go fetch the next instruction where the PC is pointing at and start over the instruction cycle.

Store Result and repeat

- Again, not all instructions uses all 6 phases:
  - Only using registers do not need compute EA.
  - Only moving data do not need execute.
- Not all (even same) instructions take the same amount of clock cycles.
  - *Cache*: a faster memory that stores frequently hit items, ins./data.
  - Cache hit rate
  - Not getting into the detail.
- ADD, for example, only takes 1 clock cycle to perform OF,EX,S in typical computers nowadays.

Instruction Pipelining

Not getting into the detail.

- Now, what if we have multiple instructions running sequentially, how many clock cycles will it take?
- Let's assume a single core/thread CPU. Meaning that, can only fetch one instruction at a time, only decode one instruction at a time, etc.
- To minimum the running time, we need to plan properly.
- **Ex.**: Ins. 1 is accessing memory at clock cycle 4, so **cannot** fetch Ins. 4.

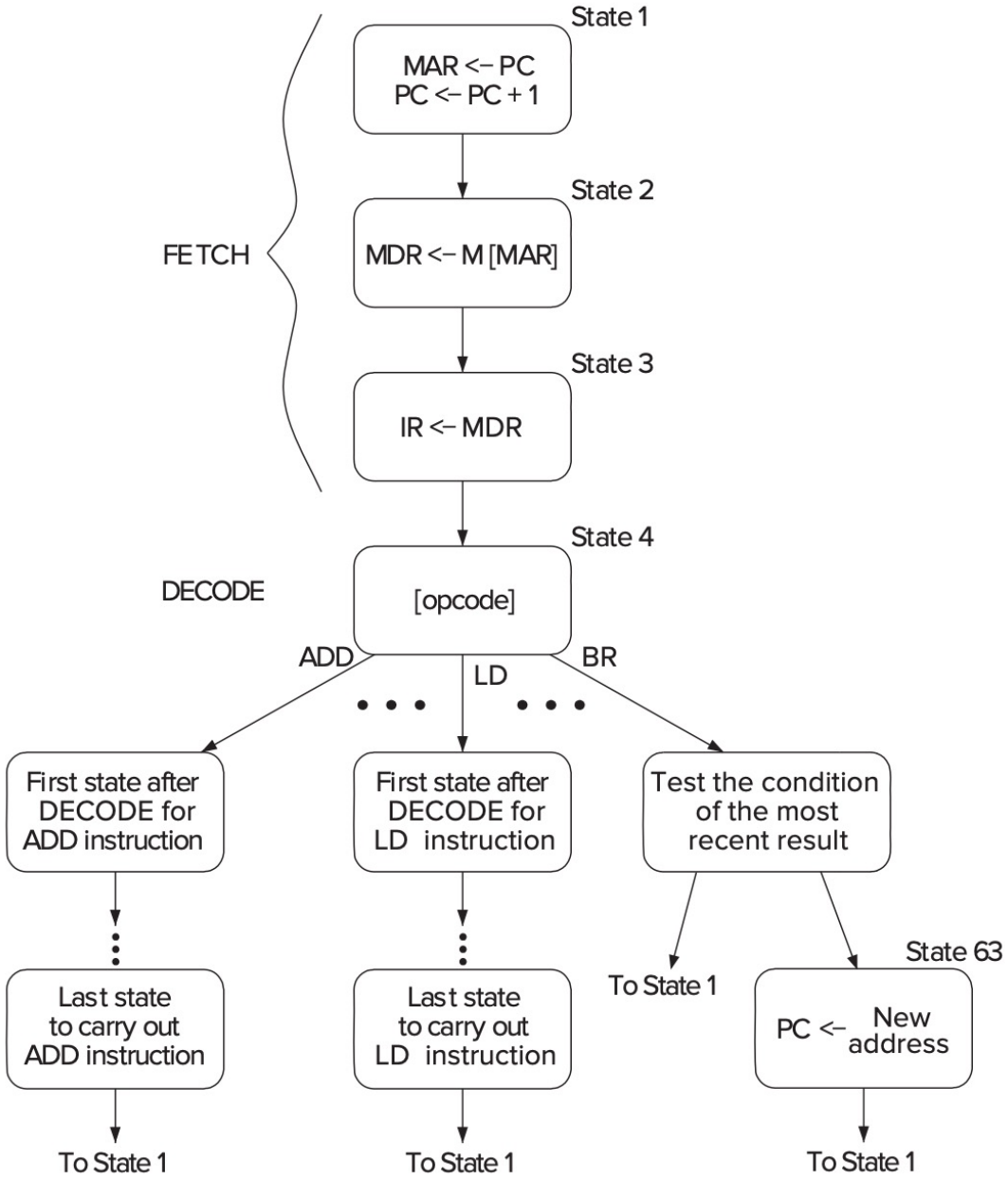
Clock Cycle	1	2	3	4	5	6	7	8
Ins. 1	IF	ID	EA	FO	EX	S		
Ins. 2		IF	ID	EA	FO	EX	S	
Ins. 3			IF	ID	EA	FO	EX	S
Ins. 4				IF				

DREW

Instruction Cycle in LC-3

- Can model the instruction cycle as a Finite State Machine that repeats; i.e., going back to State 1 and continues.

- To terminate the cycle, we HALT the clock of the *user program*.
- **Recall**: Above hardware, there is a program called *Operating System*, who manages all the user programs via CPU, memory, etc.



Questions?

- **Next Week**:
  - The LC-3 Assembly Language
  - Begin to talk about Group Project 1

- **HW 2** due on 09/27 (Fri.) by 11:59pm